

```
import pandas as pd
```

```
df=pd.read_csv('/content/drive/MyDrive/Salary Prediction/salary_data_large.csv')
```

```
print(df)
```

	Age	Gender	Education Level	Job Title \	
0	32.0	Male	Bachelor's	Software Engineer	
1	28.0	Female	Master's	Data Analyst	
2	45.0	Male	PhD	Senior Manager	
3	36.0	Female	Bachelor's	Sales Associate	
4	52.0	Male	Master's	Director	
...	...	...	...	...	...
6699	49.0	Female	PhD	Director of Marketing	
6700	32.0	Male	High School	Sales Associate	
6701	30.0	Female	Bachelor's Degree	Financial Manager	
6702	46.0	Male	Master's Degree	Marketing Manager	
6703	26.0	Female	High School	Sales Executive	

	Years of Experience	Salary
0	5.0	90000.0
1	3.0	65000.0
2	15.0	150000.0
3	7.0	60000.0
4	20.0	200000.0
...	...	...
6699	20.0	200000.0
6700	3.0	50000.0
6701	4.0	55000.0
6702	14.0	140000.0
6703	1.0	35000.0

[6704 rows x 6 columns]

```
df.head()
```

	Age	Gender	Education Level	Job Title	Years of Experience	Salary	
0	32.0	Male	Bachelor's	Software Engineer	5.0	90000.0	
1	28.0	Female	Master's	Data Analyst	3.0	65000.0	
2	45.0	Male	PhD	Senior Manager	15.0	150000.0	
3	36.0	Female	Bachelor's	Sales Associate	7.0	60000.0	
4	52.0	Male	Master's	Director	20.0	200000.0	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df.tail()
```

	Age	Gender	Education Level	Job Title	Years of Experience	Salary	
6699	49.0	Female	PhD	Director of Marketing	20.0	200000.0	
6700	32.0	Male	High School	Sales Associate	3.0	50000.0	
6701	30.0	Female	Bachelor's Degree	Financial Manager	4.0	55000.0	
6702	46.0	Male	Master's Degree	Marketing Manager	14.0	140000.0	
6703	26.0	Female	High School	Sales Executive	1.0	35000.0	

```
df.shape #(rows, columns)
```

(6704, 6)

```
df.columns
```

```
Index(['Age', 'Gender', 'Education Level', 'Job Title', 'Years of Experience',  
      'Salary'],  
      dtype='object')
```

```
df.info() #summary
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6704 entries, 0 to 6703
Data columns (total 6 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Age                   6702 non-null   float64
 1   Gender                6702 non-null   object
 2   Education Level       6701 non-null   object
 3   Job Title             6702 non-null   object
 4   Years of Experience   6701 non-null   float64
 5   Salary               6699 non-null   float64
dtypes: float64(3), object(3)
memory usage: 314.4+ KB
```

```
df.describe() #statistical summary
```

	Age	Years of Experience	Salary
count	6702.000000	6701.000000	6699.000000
mean	33.620859	8.094687	115326.964771
std	7.614633	6.059003	52786.183911
min	21.000000	0.000000	350.000000
25%	28.000000	3.000000	70000.000000
50%	32.000000	7.000000	115000.000000
75%	38.000000	12.000000	160000.000000
max	62.000000	34.000000	250000.000000

```
#Handeling Missing Data in Pandas
```

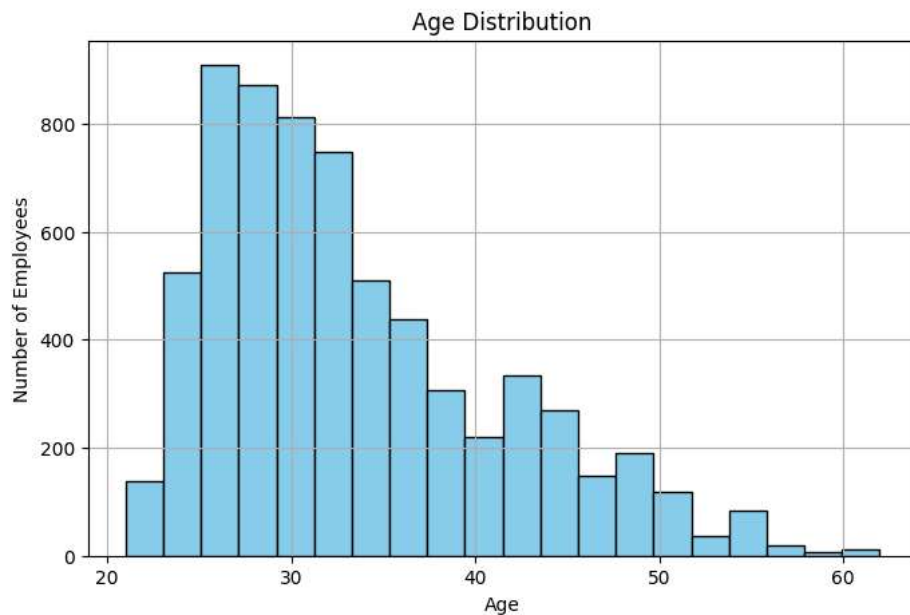
```
df.dropna(inplace=True) #remove rows with missing data
df.fillna(0) #fill NaNs with 0
```

	Age	Gender	Education Level	Job Title	Years of Experience	Salary
0	32.0	Male	Bachelor's	Software Engineer	5.0	90000.0
1	28.0	Female	Master's	Data Analyst	3.0	65000.0
2	45.0	Male	PhD	Senior Manager	15.0	150000.0
3	36.0	Female	Bachelor's	Sales Associate	7.0	60000.0
4	52.0	Male	Master's	Director	20.0	200000.0
...	...	...	...	...	...	...
6699	49.0	Female	PhD	Director of Marketing	20.0	200000.0
6700	32.0	Male	High School	Sales Associate	3.0	50000.0
6701	30.0	Female	Bachelor's Degree	Financial Manager	4.0	55000.0
6702	46.0	Male	Master's Degree	Marketing Manager	14.0	140000.0
6703	26.0	Female	High School	Sales Executive	1.0	35000.0

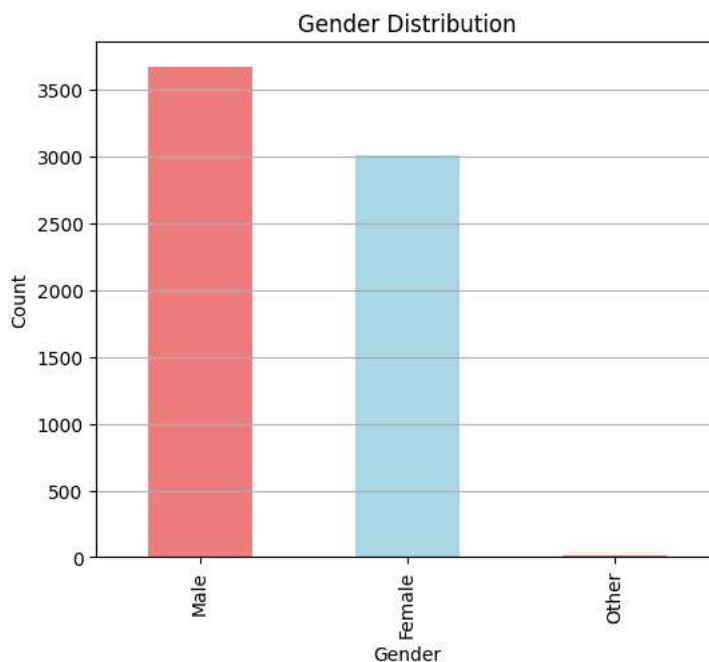
6698 rows × 6 columns

```
import matplotlib.pyplot as plt
```

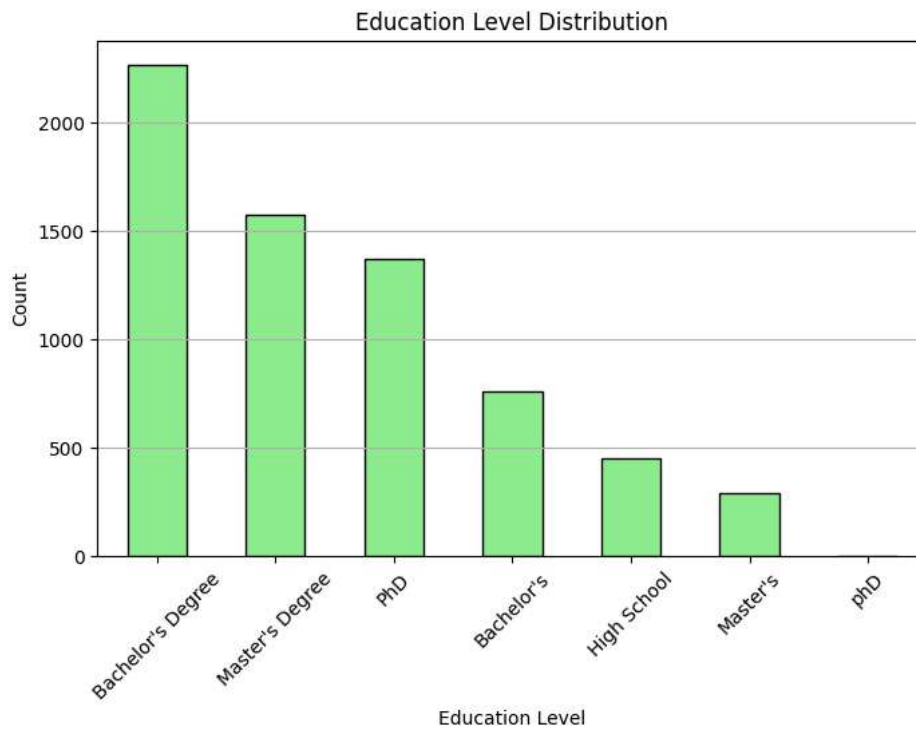
```
# 1 Distribution of Age
plt.figure(figsize=(8, 5))
plt.hist(df['Age'], bins=20, color='skyblue', edgecolor='black')
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Number of Employees")
plt.grid(True)
plt.show()
```



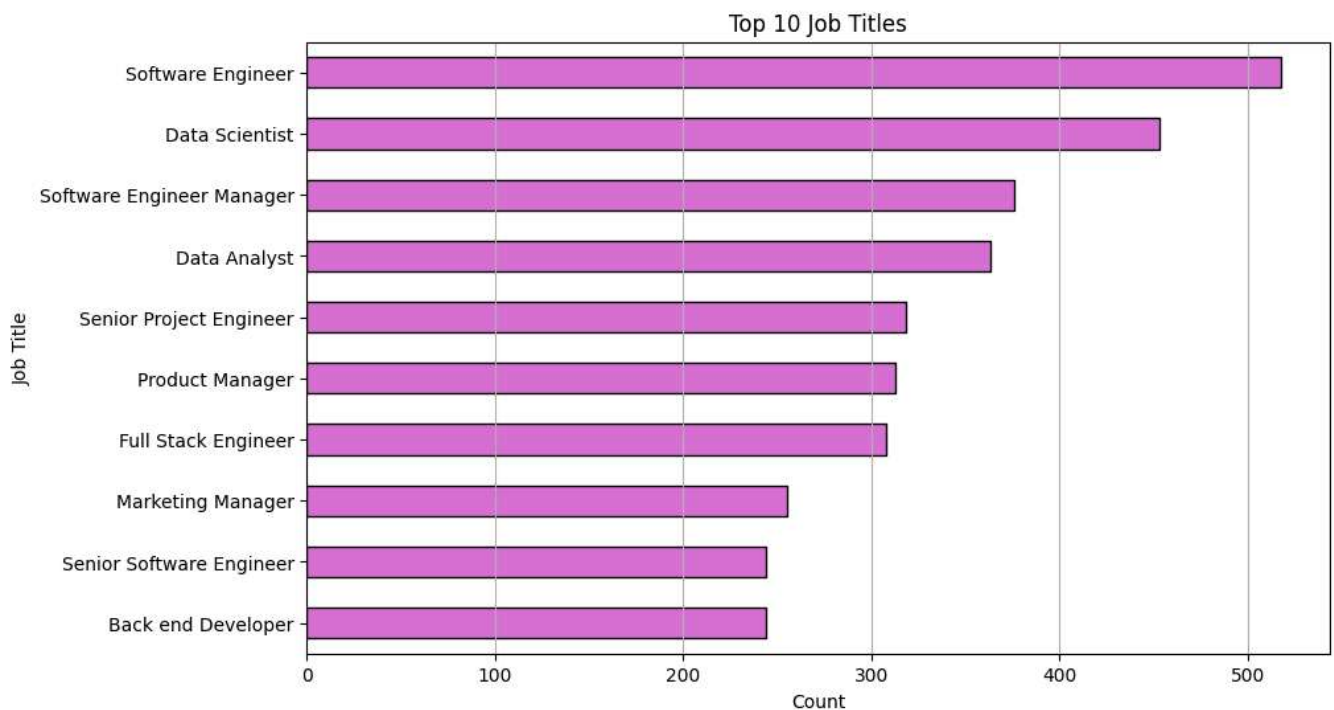
```
# 2 Gender Distribution
plt.figure(figsize=(6, 5))
df['Gender'].value_counts().plot(kind='bar', color=['lightcoral', 'lightblue'])
plt.title("Gender Distribution")
plt.xlabel("Gender")
plt.ylabel("Count")
plt.grid(axis='y')
plt.show()
```



```
# 3 Education Level Distribution
plt.figure(figsize=(8, 5))
df['Education Level'].value_counts().plot(kind='bar', color='lightgreen', edgecolor='black')
plt.title("Education Level Distribution")
plt.xlabel("Education Level")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.grid(axis='y')
plt.show()
```

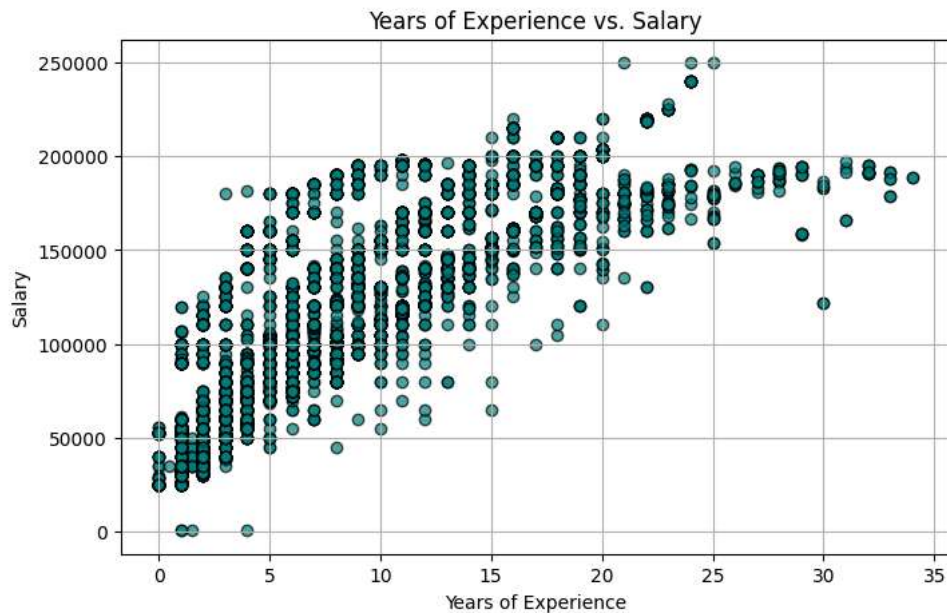


```
# 4 Job Title Distribution (Top 10)
plt.figure(figsize=(10, 6))
df['Job Title'].value_counts().head(10).plot(kind='barh', color='orchid', edgecolor='black')
plt.title("Top 10 Job Titles")
plt.xlabel("Count")
plt.ylabel("Job Title")
plt.gca().invert_yaxis()
plt.grid(axis='x')
plt.show()
```

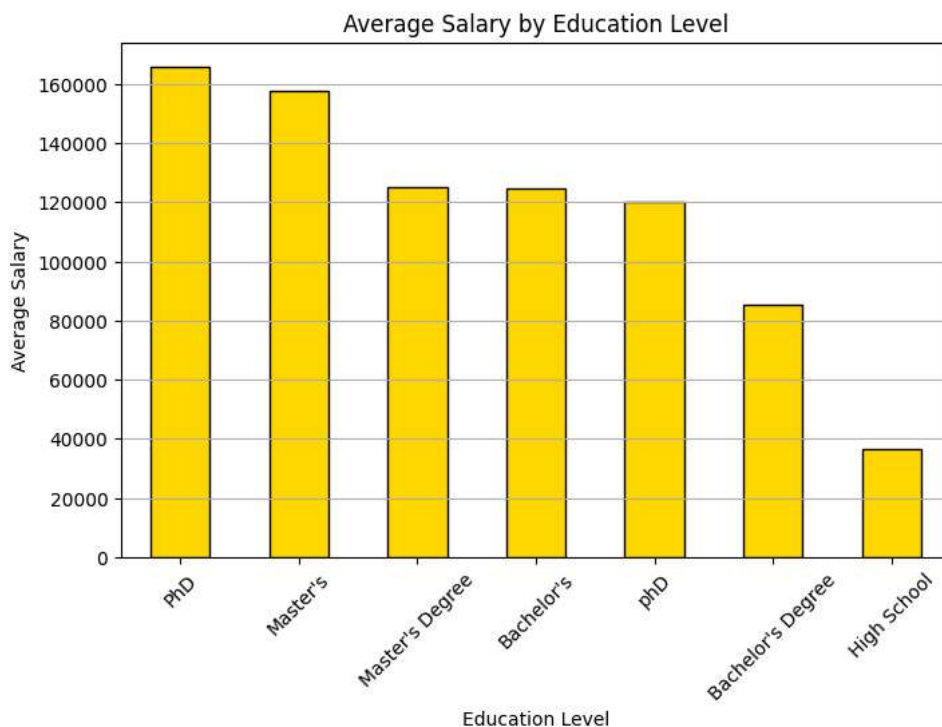


```
# 5 Years of Experience vs. Salary
plt.figure(figsize=(8, 5))
plt.scatter(df['Years of Experience'], df['Salary'], alpha=0.7, color='teal', edgecolors='black')
plt.title("Years of Experience vs. Salary")
plt.xlabel("Years of Experience")
```

```
plt.ylabel("Salary")
plt.grid(True)
plt.show()
```



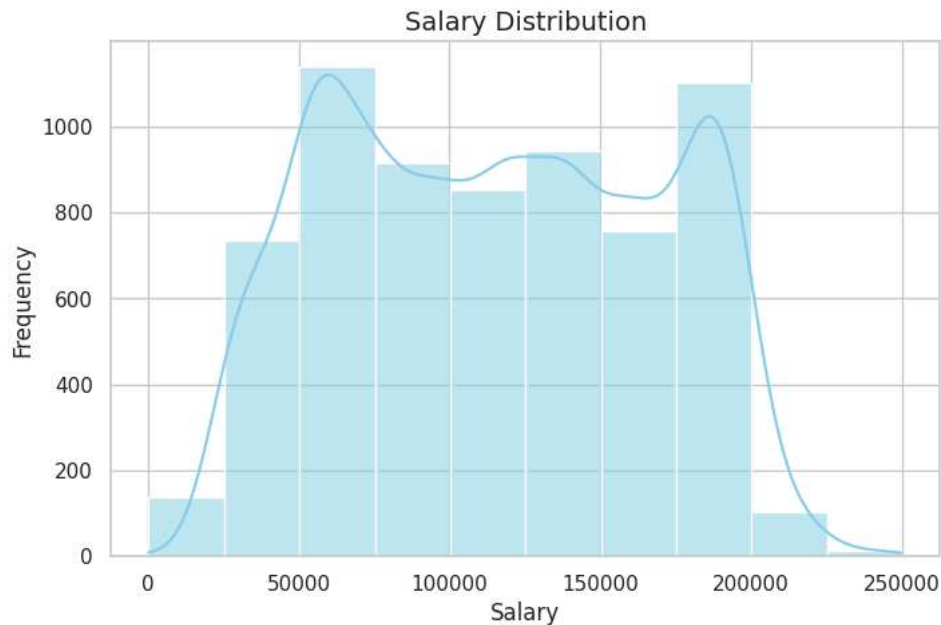
```
# 6 Average Salary by Education Level
plt.figure(figsize=(8, 5))
df.groupby('Education Level')['Salary'].mean().sort_values(ascending=False).plot(kind='bar', color='gold', edgecolor='black')
plt.title("Average Salary by Education Level")
plt.xlabel("Education Level")
plt.ylabel("Average Salary")
plt.xticks(rotation=45)
plt.grid(axis='y')
plt.show()
```



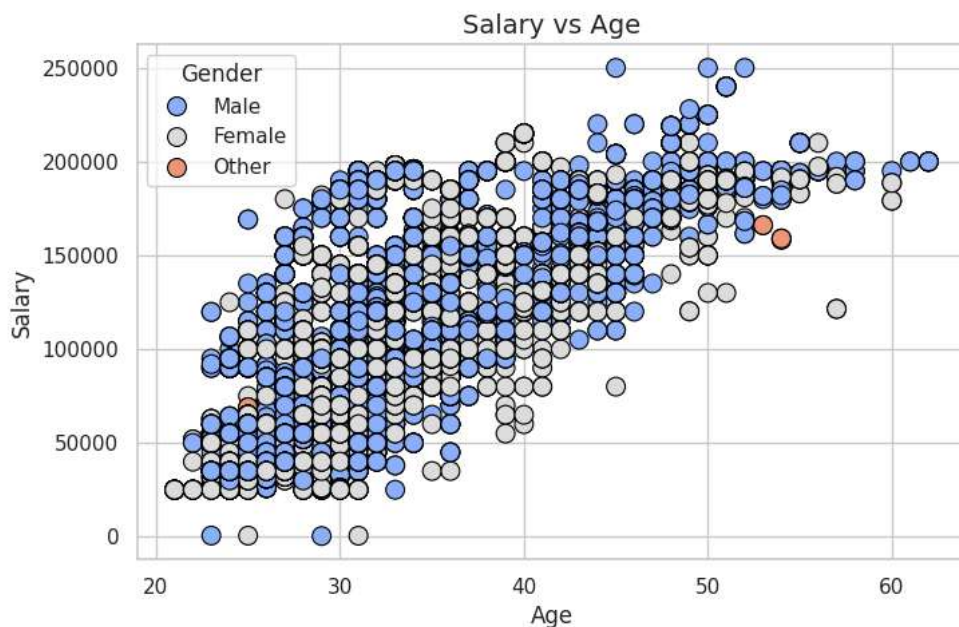
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Optional styling
sns.set(style="whitegrid")
```

```
# 1 Salary Distribution
# -----
plt.figure(figsize=(8, 5))
sns.histplot(df['Salary'], bins=10, kde=True, color='skyblue')
plt.title("Salary Distribution", fontsize=14)
plt.xlabel("Salary")
plt.ylabel("Frequency")
plt.grid(True)
plt.show()
```



```
# 2 Salary vs Age
# -----
plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x='Age', y='Salary', hue='Gender', palette='coolwarm', s=100, edgecolor='black')
plt.title("Salary vs Age", fontsize=14)
plt.xlabel("Age")
plt.ylabel("Salary")
plt.legend(title="Gender")
plt.grid(True)
plt.show()
```

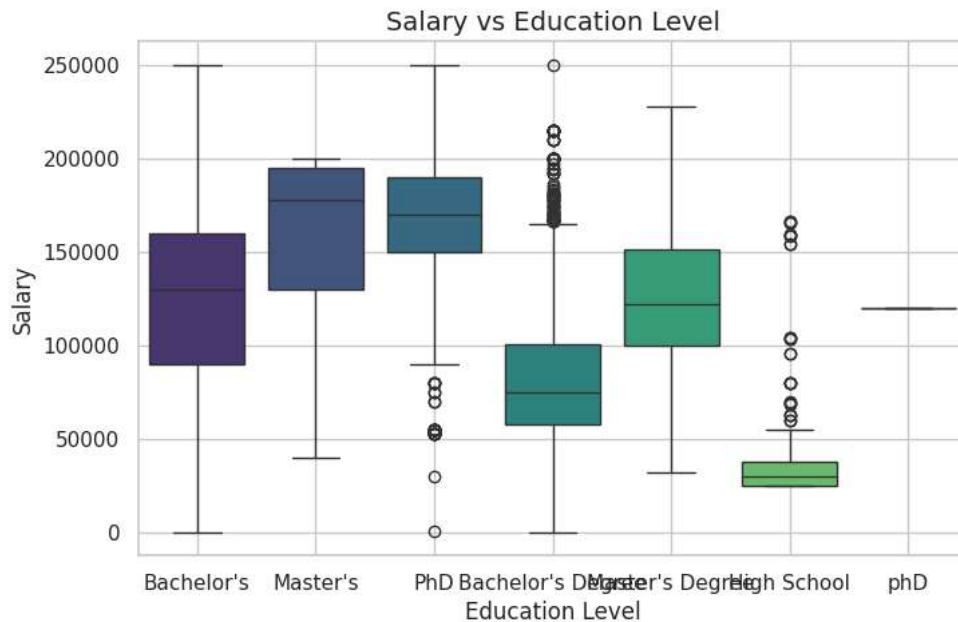


```
# 3 Salary vs Education Level
# -----
plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x='Education Level', y='Salary', palette='viridis')
plt.title("Salary vs Education Level", fontsize=14)
plt.xlabel("Education Level")
plt.ylabel("Salary")
plt.grid(True)
plt.show()
```

/tmp/ipython-input-2787493307.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `l`

```
sns.boxplot(data=df, x='Education Level', y='Salary', palette='viridis')
```



```
# 4 Salary vs Years of Experience
# -----
plt.figure(figsize=(8, 5))
sns.regplot(data=df, x='Years of Experience', y='Salary',
            scatter_kws={'alpha': 0.7, 'color': 'teal'}, line_kws={'color': 'red'})
plt.title("Salary vs Years of Experience", fontsize=14)
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.grid(True)
plt.show()
```

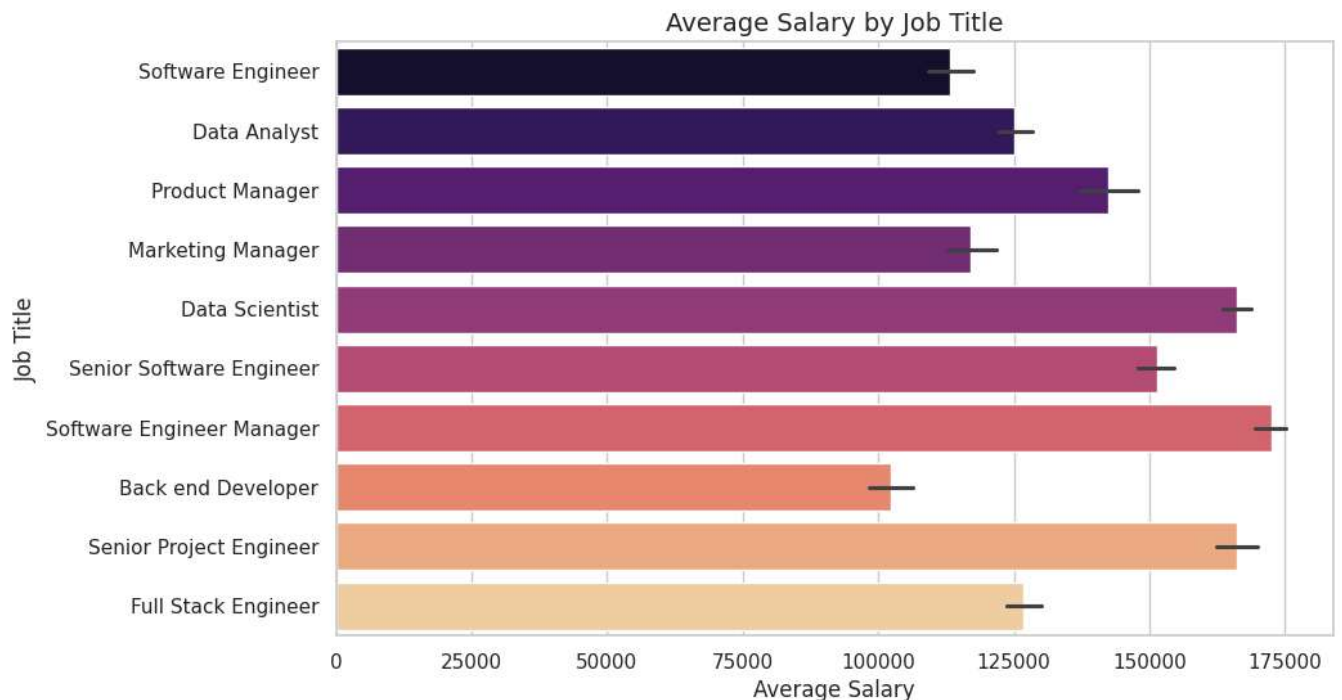


```
# 5 Salary vs Job Title (Top 10)
# -----
plt.figure(figsize=(10, 6))
top_jobs = df['Job Title'].value_counts().head(10).index
sns.barplot(data=df[df['Job Title'].isin(top_jobs)],
            x='Salary', y='Job Title', palette='magma')
plt.title("Average Salary by Job Title", fontsize=14)
plt.xlabel("Average Salary")
plt.ylabel("Job Title")
plt.grid(True, axis='x')
plt.show()
```

/tmp/ipython-input-466915057.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `l

```
sns.barplot(data=df[df['Job Title'].isin(top_jobs)],
```



```
# -----
# 6 Average Salary by Gender
# -----
plt.figure(figsize=(6, 5))
sns.barplot(data=df, x='Gender', y='Salary', estimator='mean', palette=['lightcoral', 'lightblue'])
plt.title("Average Salary by Gender", fontsize=14)
plt.xlabel("Gender")
plt.ylabel("Average Salary")
plt.grid(True, axis='y')
plt.show()
```

/tmp/ipython-input-2819322205.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `l

```
sns.barplot(data=df, x='Gender', y='Salary', estimator='mean', palette=['lightcoral', 'lightblue'])
/tmp/ipython-input-2819322205.py:5: UserWarning:
The palette list has fewer values (2) than needed (3) and will cycle, which may produce an uninterpretable plot.
sns.barplot(data=df, x='Gender', y='Salary', estimator='mean', palette=['lightcoral', 'lightblue'])
```



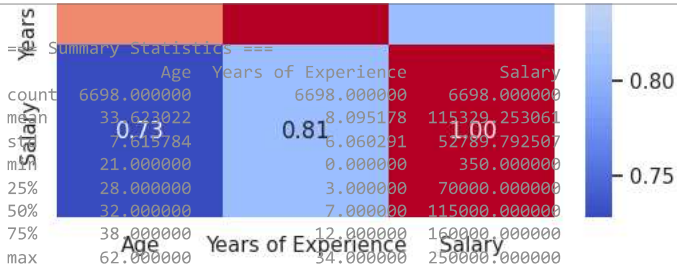
```
# 7 Correlation Heatmap (Numerical Columns)
# -----
plt.figure(figsize=(6, 5))
sns.heatmap(df[['Age', 'Years of Experience', 'Salary']].corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap", fontsize=14)
plt.show()
```

Correlation Heatmap

```
# ☒ Summary printout
# -----
print("\n=== Summary Statistics ===")
print(df[['Age', 'Years of Experience', 'Salary']].describe())

print("\n=== Average Salary by Education Level ===")
print(df.groupby('Education Level')['Salary'].mean())

print("\n=== Average Salary by Job Title ===")
print(df.groupby('Job Title')['Salary'].mean().sort_values(ascending=False))
```



```
=== Average Salary by Education Level ===
Education Level
Bachelor's          124767.658730
Bachelor's Degree   85174.886093
High School         36706.694196
Master's           157604.166667
Master's Degree    125075.333969
PhD                165684.828947
phD                120000.000000
Name: Salary, dtype: float64
```

```
=== Average Salary by Job Title ===
```